The Major Gaps:

- **Memory & RAG:** Your pitch promises MemGPT (LRU paging) and GraphRAG/Bi-Temporal Knowledge. The PDF explicitly parks GraphRAG, temporal knowledge graphs, and MemGPT into the "P2 (cut without guilt)" tier. It forces you to build a simple Vector DB and standard text chunking first.
- **Tooling:** Your pitch relies on ScaleMCP for dynamic tool binding. The PDF restricts you to a hardcoded minimum of 6 tools (read_document, search_knowledge, read_csv, run_python, generate_docx, human_approval) to guarantee execution.
- **Orchestration:** Your pitch uses a complex AutoGen 3-Agent pattern. The PDF mandates a simpler, rule-based router (not learned) and a single bounded agent state machine (INTAKE, PLAN, RETRIEVE, ACT, OBSERVE, VERIFY).
- **Hardware:** The PDF assumes you have a single 16-24GB VRAM workstation. Running a 7B-8B model alongside a Vision model on a single 6GB RTX 4050 will instantly result in Out-Of-Memory (OOM) crashes.

The Hardware Blueprint: 2x 6GB GPUs + 1 MacBook

To survive with two 6GB RTX 4050s and a MacBook, you must distribute the compute. You cannot run everything on one machine.

- **Laptop 1 (RTX 4050 - The Brain):** Install **Ollama**. Load Llama-3.1-8B-Instruct quantized to 4-bit (GGUF). This will consume ~5.5GB of VRAM. This laptop serves as the local API exclusively for general reasoning and routing.
- **Laptop 2 (RTX 4050 - The Engine):** Install **Ollama**. Load Qwen-2.5-Coder-7B and Qwen-2.5-VL-7B (both 4-bit). You will hot-swap these models in RAM depending on whether the task is a calculation or a P&ID scan.
- **The MacBook (The Orchestrator):** MacBooks excel at CPU-bound tasks and web serving. Run your Next.js/React frontend, FastAPI backend, vector database (Qdrant/FAISS), and the Docker Sandbox on this machine. Route API calls from the MacBook to the local IP addresses of Laptop 1 and 2.

Web-Based vs. App-Based

This must be a **Web-Based Desktop Portal**.

- **The Enterprise Reality:** MRPL maintenance engineers do not write multi-page mathematical approval notes or analyze massive, highly detailed P&ID blueprints on a 6-inch mobile screen.
- **The PDF Alignment:** The execution plan explicitly calls for a "browser file upload backend" and a frontend shell built with Node, React, and Tailwind, featuring a side navigation bar (Dashboard, Documents, AI Jobs, Artifacts, etc.).

- **The Sandbox:** Generating .docx deliverables and executing Python inside an isolated Docker container is a server-side/desktop operation, entirely unsuited for a mobile app deployment.

Project Blueprint & Where to Start

🔴 **Start Here: Day 0 - Infrastructure & Data Freeze**

Before writing a single line of application logic, you must establish the offline environment.

1. Network the MacBook and both laptops via a local offline switch or a mobile hotspot with mobile data turned off.
2. Test API calls from the MacBook to the Ollama instances on both RTX 4050 laptops. The exit criteria is that offline startup works and every model loads once with the internet off.
3. **Crucial:** Build the demo_data/ folder today. You need an inspection_report.pdf, maintenance_history.csv, a sample_pid.png, and a .docx report template.

Phase 1: The UI and The Router (Day 1 & Day 2)

- Build the React frontend shell with the upload box and sidebar.
- Establish the FastAPI endpoints (POST /upload, POST /task).
- Implement the rule-based router: If the user uploads a PDF, route the API call to Laptop 1 (Llama 3.1). If they upload an image/P&ID, route to Laptop 2 (Qwen-VL).

Phase 2: Knowledge & Sandbox (Day 3 & Day 4)

- Build the simple RAG pipeline on the MacBook using a local vector DB to extract text and answer questions with grounded sources.
- Set up the Docker sandbox with network_mode: none and pre-bake pandas and python-docx.
- Wire the Coder model (Laptop 2) to generate scripts that compile the final Word document.

Phase 3: The Security Shield (Day 5)

- Implement the eBPF/psutil live network monitor on the MacBook to prove zero external calls and zero outbound traffic.
- Create the audit log and SHA-256 hashing for input/output files. Ensure you can physically unplug the router mid-demo while the UI continues to function perfectly.